LABSHEET - 10

CS F211 - Data Structures and Algorithms
Academic Year: 2025-2026 (2nd Semester)

General Instructions for this Labsheet:

- Use of any built-in function like qsort(), bsearch(), sqrt(), etc. is not allowed.
- All the questions should be done by Hashing or Divide and Conquer.

Consider **0-based** indexing whenever needed.

Question 1: Sorted Intersector (Chaining with Ordered Insert)

Description: A data broker needs to find user profiles that exist in two separate databases. To optimize the search, implement a hash table of size m using separate chaining where every linked list must remain strictly sorted in ascending order.

1. First, insert all UserIDs from Database A into the hash table using the hash function h(k) = k % m.
2. When inserting into a chain, the new UserID must be placed in its correct sorted position (ascending). Assume all UserIDs in Database A are strictly unique.
3. Next, iterate through the UserIDs in Database B. Search the hash table for each ID. Assume all UserIDs in Database B are also strictly unique. If the ID is found, add it to your intersection list.

Input Format:

- The first line contains three space-separated integers: m (hash table size), nA (size of Database A), and nB (size of Database B).
- The second line contains nA space-separated integers for Database A.
- The third line contains nB space-separated integers for Database B.

Output Format:

- Print Intersection: followed by the matching UserIDs in the **exact order** they were searched in Database B.
- If no IDs match, print Intersection: None.

Sample Test Case 1:
Input:
```
7 8 6
42 15 8 22 99 12 7 35
15 10 7 42 100 8
```
Output:
```
Intersection: 15 7 42 8
(Explanation: At index 1, the elements 15, 8, 22, and 99 are
inserted. Because of the sorted insertion rule, the chain at index 1
internally becomes 8 -> 15 -> 22 -> 99)
```

Sample Test Case 2:
Input:
```
5 6 5
10 20 30 40 50 60
15 25 35 45 55
```
Output:
```
Intersection: None
```

Question 2: VIP Warehouse Allocation (Priority Based Linear Probing)

Description: A warehouse has m consecutive bays (indexed 0 to m-1). Containers are assigned using linear probing: (h(k) + i) mod m. Empty bays are initialized to -1. There are two types of containers: Regular (ID < 1000) and VIP (ID >= 1000). You will be given exactly n containers to insert. It is guaranteed that n ≤ m, meaning the warehouse will never overflow.

When a container arrives, the system attempts to place it starting at index h(k) = k mod m:

1. If the current index is empty (-1), the container is placed.
2. If the current index holds a Regular container, and the inserting container is a VIP, the VIP container swaps with the Regular container. The VIP takes the bay, and the kicked-out Regular container immediately becomes the "inserting container", resuming its probe from the very next index: (current_index + 1) % m.
3. If no swap occurs (e.g., Regular meets Regular, VIP meets VIP, or Regular meets VIP), the inserting container simply checks the next index: (current_index + 1) % m.

Input Format:

- The first line contains two space-separated integers: m (total bays) and n (number of containers to insert).
- The second line contains n space-separated integers representing the ContainerIDs to be inserted in the order they arrive.

Output Format:

- Print the final state of the warehouse array as m space-separated integers on a single line.

Sample Test Case 1:
Input:
```
7 6
15 22 8 1001 1008 1022
```
Output:
```
1001 1008 1022 8 15 22 -1
(Explanation: 1008 hashes to 0, collides with VIP 1001, moves to 1 and kicks
out Regular 15. 15 probes forward and settles at index 4. Later, 1022 hashes to
0, probes past the VIPs to index 2, kicks out Regular 22. 22 probes forward and
settles at index 5.)
```
Sample Test Case 2:
Input:
```
5 5
10 15 1000 1005 12
```
Output:
```
1000 1005 10 15 12
(Explanation: 10 goes to 0. 15 goes to 1. 1000 (VIP) goes to 0, kicks out 10.
10 moves to 2. 1005 (VIP) goes to 0, moves to 1, kicks out 15. 15 moves to 3.
12 hashes to 2, moves past 10 and 15, and settles at 4.)
```

Question 3: E-Commerce Shopping Cart (Implementing a Map)

Description: Implement a Map (Key-Value data structure) using separate chaining.

- **Key:** ProductID (always >= 0).
- **Value:** Quantity (always > 0).
- **Hash function:** h(ProductID) = ProductID % m.

Process a series of cart operations:

- 1 ProductID Qty: Add the Qty to the cart. Iterate through the chain at the hashed index. If the ProductID exists, update its value by adding the new Qty. If the ProductID is not found after traversing the entire chain, insert a new Key-Value node at the strict tail of the chain.
- 2: Print the current state of the map.

Input Format:

- The first line contains m (hash table size) and q (number of queries).
- The next q lines contain queries structured as either 1 ProductID Qty or 2.

Output Format:

- For every query 2, print exactly m lines.
- Format each node exactly as [Key:Value]. Connect multiple nodes with ->.
- Print Empty if the index has no nodes.

Sample Test Case 1:
Input:
```
5 8
1 12 5
1 22 3
1 7 10
1 12 4
2
1 8 1
1 7 5
2
```
Output:
```
Index 0: Empty
Index 1: Empty
Index 2: [12:9] -> [22:3] -> [7:10]
Index 3: Empty
Index 4: Empty
Index 0: Empty
Index 1: Empty
Index 2: [12:9] -> [22:3] -> [7:15]
Index 3: [8:1]
Index 4: Empty
```

Sample Test Case 2:
Input:
```
3 6
1 3 100
1 6 50
1 9 25
1 1 10
1 4 20
2
```
Output:
```
Index 0: [3:100] -> [6:50] -> [9:25]
Index 1: [1:10] -> [4:20]
Index 2: Empty
```

Question 4: Double Hashing

Problem Description:
Implement a hash table of size **M** that resolves collisions using double hashing. You will be given a sequence of N integer keys to insert into the table.

Primary hash function is: **`h1(k) = k mod M`**
Secondary hash function is: **`h2(k) = R - (k mod R)`**

The probe sequence for inserting a key k is defined as:
`h(k, i) = (h1(k) + i * h2(k)) mod M`
where i >= 0 is the probe number starting at 0.
Note: 'i' is a Probe Counter. It will be incremented by 1 whenever a collision occurs for a key.

Input Format:
Line 1: Two integers M, R (Size of hash table M, a constant R; M, R are prime Numbers, R < M)
Line 2: One integer (N, number of elements to insert, N < M)
Line 3: N integers (Keys to be inserted)

Output Format:
M integers representing the final state of the hash table. Use -1 to denote an empty slot.

Sample Test Case 1:

Input:
```
11 7
4
10 21 32 12
```

Output:
```
-1 12 32 -1 -1 -1 21 -1 -1 -1 10
```

Explanation:
*** Insert 10:** h1(10) = 10 mod 11 = 10. Index 10 is empty. Insert at 10. *** Insert 21:** h1(21) = 21 mod 11 = 10. Collision at 10. Calculate h2(21) = 7 - (21 mod 7) = 7.
i=1: (10 + 1*7) mod 11 = 17 mod 11 = 6. Index 6 is empty. Insert at 6.
*** Insert 32:** h1(32) = 32 mod 11 = 10. Collision at 10. Calculate h2(32) = 7 - (32 mod 7) = 3.
i=1: (10 + 1*3) mod 11 = 13 mod 11 = 2. Index 2 is empty. Insert at 2.
*** Insert 12:** h1(12) = 12 mod 11 = 1. Index 1 is empty. Insert at 1.

Sample Test Case 2:

Input:

```
13 11
5
14 27 40 53 20
```

Output:

```
-1 14 -1 53 -1 40 -1 27 -1 20 -1 -1 -1
```

Explanation:z

*** Insert 14:** h1(14) = 14 mod 13 = 1. Index 1 is empty. Insert at 1.
*** Insert 27:** h1(27) = 27 mod 13 = 1. Collision at 1.
Calculate h2(27) = 11 - (27 mod 11) = 6.
i=1: (1 + 1 * 6) mod 13 = 7 mod 13 = 7. Index 7 is empty. Insert at 7. *** Insert 40:** h1(40) = 40 mod 13 = 1. Collision at 1.
Calculate h2(40) = 11 - (40 mod 11) = 4.
i=1: (1 + 1 * 4) mod 13 = 5 mod 13 = 5. Index 5 is empty. Insert at 5. *** Insert 53:** h1(53) = 53 mod 13 = 1. Collision at 1.
Calculate h2(53) = 11 - (53 mod 11) = 2.
i=1: (1 + 1 * 2) mod 13 = 3 mod 13 = 3. Index 3 is empty. Insert at 3. *** Insert 20:** h1(20) = 20 mod 13 = 7. Collision at 7.
Calculate h2(20) = 11 - (20 mod 11) = 2.
i=1: (7 + 1 * 2) mod 13 = 9 mod 13 = 9. Index 9 is empty. Insert at 9.

Question 5: Lazy Deletion in Linear Probing

Problem Description:
Implement a hash table of size m using linear probing to resolve collisions. Empty slots are initialized with -1. In addition to insertion, the system must support lazy deletion using a special marker value -2 (indicating a deleted slot).

Process the following operations:
- 1 X: Insert key X into the hash table using linear probing.
- 2 X: Delete key X from the hash table. Replace the slot with -2 if found. Otherwise do nothing.
- 3 X: Search for key X and print Found or Not Found.

Rules:
- While inserting, treat -2 (deleted slots) as valid positions for insertion.
- While searching, probing must continue past -2 slots.

Hash function: h(k) = k % m

Input Format:
Line 1: Two integers m (hash table size) and q (number of queries)
Line 2 to q+1: Each line contains a query of type 1 X, 2 X, or 3 X.
Constraints:
X >=0

Output Format:
For every query of type 3, print Found or Not Found on a new line.
After processing all queries, print the final state of the hash table.

Sample Test Case 1:

Input:
```
7 6
1 10
1 17
1 24
2 17
1 31
3 17
```

Output:
```
Not Found
10 -2 24 31 -1 -1 -1
```

Not Found
-1 -1 -1 10 31 24 -1

Sample Test Case 2:

Input:
```
5 5
1 5
1 10
2 5
3 10
3 5
```

Output:
```
Found
Not Found
-2 10 -1 -1 -1
```

Question 6: Most Frequent Element using Chaining

Problem Description:
You are given a sequence of N integers. Implement a hash table of size m using separate chaining to count the frequency of each element.
Hash function: h(k) = k % m

Rules:
- Each chain stores nodes of the form [Key:Frequency].
- If a key already exists in the chain, increment its frequency.
- Otherwise, insert it at the tail of the chain.

After processing all elements, determine the most frequent element.
If multiple elements have the same maximum frequency, print the smallest such element.
Additionally, print the final state of the hash table.

Input Format:
Line 1: Two integers m and N (hash table size and number of elements)
Line 2: N integers (input sequence)

Output Format:
1. Print exactly m lines representing the hash table.
Format each node exactly as [Key:Frequency]. Connect multiple nodes with ->.
Print Empty if the index has no nodes.
2. Print two integers:
Element and Frequency

Sample Test Case 1:
Input:
```
5 8
10 15 10 20 15 10 25 15
```

Output:
```
Index 0: [10:3] -> [15:3] ->
[20:1] -> [25:1]
Index 1: Empty
Index 2: Empty
Index 3: Empty
Index 4: Empty
10 3
```

Sample Test Case 2:
Input:
```
4 6
7 11 7 11 3 3
```

Output:
```
Index 0: Empty
Index 1: [7:2] -> [11:2] ->
[3:2]
Index 2: Empty
Index 3: Empty
3 2
```

wrong

Question 7: Skip List Search Path Length

Problem Description:
You are required to implement a **deterministic Skip List** to store N distinct integers.
The skip list must be constructed using the following rule:

- Level 0 contains all elements in sorted order
- Level i (i > 0) contains every 2^i-th element from Level 0 (starting from index 0)

After constructing the skip list, process Q search queries.
For each search query X, determine the number of steps taken to search for X in the skip list.

Definition of a step:

- Moving right to the next node counts as 1 step
- Moving down a level counts as 1 step

Rules:

- Searching always starts from the top-most level at the smallest element
- If X is found, print the number of steps taken
- If X is not found, print -1

Input Format:
Line 1: Two integers N and Q
Line 2: N integers (elements to insert into skip list)
Line 3: Q integers (search queries)

Output Format:
Print Q integers representing the steps taken for each query (or -1 if not found)

Sample Test Case 1:

Input:
5 3
10 20 30 40 50
20 35 50

Output: something is wrong
3 -1 4

Sample Test Case 2:

Input:
6 2
5 15 25 35 45 55
5 60

Output:
2 -1

Question 8: 2x Inversions

Problem Description:
Given an integer array A of size N, return the number of pairs `(i, j)` in the given array satisfying the following condition:

- `0 <= i < j < n`
- `A[i] > 2 * A[j]`

Note: Consider **0-based** indexing
Note: You have to do the question using **Divide and Conquer Algorithm ONLY**.

Input Format:
Line 1: One integer N (Length of array, N ≥ 3)
Line 2: N integers (Array A elements)

Output Format:
Total number of such pairs

Sample Test Case 1:
Input:
```
6
1 3 2 3 1 1
```

Output:
```
4
```

Sample Test Case 2:

Input:
```
8
5 2 7 6 2 5 1 3
```

Output:
```
9
```

Question 9: City Skyline

Problem Description:
A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance.
Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given as an array of three values:

```
buildings[i] = [left_i, right_i, height_i]
```

- `left_i` is the x coordinate of the left edge of the ith building
- `right_i` is the x coordinate of the right edge of the ith building
- `height_i` is the height of the ith building.

The skyline should be represented as a list of "key points" sorted by their x-coordinate.
A key point is the left endpoint of a horizontal line segment.
The last key point always has a y-coordinate of 0 and is used to indicate the rightmost edge of the skyline.

Note: Consider **0-based** indexing
Note: You have to do the question using **Divide and Conquer Algorithm ONLY**.
Note: The input buildings are sorted by `left_i` in non-decreasing order.

Input Format:
Line 1: One integer N (Number of buildings, N ≥ 3)
Next N Lines: 3 integers (`left_i, right_i, height_i` for each building)

Output Format:
Print the key points of the skyline, sorted by their x-coordinate.
Each key point should be printed on a new line as two space-separated integers representing the x and y coordinates.

skip

Sample Test Case 1:

Input:

```
4
2 9 10
3 7 15
5 12 12
15 20 10
```

Output:

```
2 10
3 15
7 12
12 0
15 10
20 0
```

Sample Test Case 2:

Input:

```
6
1 5 11
2 7 6
3 9 13
12 16 7
14 25 3
19 22 18
```

Output:

```
1 11
3 13
9 0
12 7
16 3
19 18
22 3
25 0
```

Question 10: Find String in String

Problem Description:
You are given two strings: T, P

Find all the occurrences of string P in T and print the corresponding starting indices.

Note: Consider **0-based** indexing
Note: Both the strings will contain elements between a-z.
Note: Solve this using the **String Rolling Hash** algorithm **ONLY** (Explained below).
Note: You have to solve this in O(N + M) Time complexity.

Input Format:
Line 1: Two integers N, M (Length of string T and P, N ≥ M)
Line 2: N characters (string T)
Line 3: M characters (string P)

Output Format:
Print all the indices where P can be found in T

Sample Test Case 1:

Input:
```
6 3
abbaab
ab
```

6 2

Output:
```
0 4
```

Explanation:
```
abbaab -> 0
abbaab -> 4
```

Sample Test Case 2:

Input:
```
8
abababab
aba
```

8 3

Output:
```
0 2 4
```

Explanation:
```
ababab ab -> 0
ababab ab -> 2
ababab ab -> 4
```

String Rolling Hash algorithm

For the given problem, we will calculate a **unique hash value** for the **string P**.
And then, we will check if the **same hash value** appears anywhere in the **string T** or not.

For, this, we will choose:
Base = 29
MOD = 1e9 + 7 (a big prime number)

Let's assume **P = abc, T = abcaabc**
We will **map all characters** [a-z] **to numbers** [1-26]

Hash value of P -

Hp = ((a * (Base^0)) + (b * (Base^1)) + (c * (Base^2))) % MOD

This will give us a unique hash value for string P.

Hash array for string T -

For all the windows of size **m = len(P)**, calculate the Hash Values for them similarly, and add them to an **array A** as shown:

A[i] = (A[i-1] - (T[i-1] * Base^(i-1)) + (T[i+m-1] * Base^(i+m-1)) + MOD) % MOD

A[0] = ((a * (Base^0)) + (b * (Base^1)) + (c * (Base^2))) % MOD
A[1] = ((b * (Base^1)) + (c * (Base^2)) + (a * (Base^3))) % MOD
A[2] = ((c * (Base^2)) + (a * (Base^3)) + (a * (Base^4))) % MOD
A[3] = ((a * (Base^3)) + (a * (Base^4)) + (b * (Base^5))) % MOD
A[4] = ((a * (Base^4)) + (b * (Base^5)) + (c * (Base^6))) % MOD

Then sequentially check if:
(Hp)*(Base^0) = A[0]
(Hp)*(Base^1) = A[1]
(Hp)*(Base^2) = A[2]
(Hp)*(Base^3) = A[3]
(Hp)*(Base^4) = A[4]

Output all the indices for which these values match.
Overall Time Complexity: **O(M + N)**